

RescueRight MVP Integration Guide

ESP32 + 4 MPU6500 → React Native App

Version: 1.0 Final

Date: October 14, 2025

Hardware: ESP32 DevKit V1 + 4x MPU6500 IMU sensors on foam

Goal: Working prototype with real-time feedback



Table of Contents

1. [System Overview](#)
2. [Hardware Setup](#)
3. [Understanding the Physics](#)
4. [ESP32 Firmware](#)
5. [Testing & Validation](#)
6. [App Integration](#)
7. [Calibration Process](#)
8. [Troubleshooting](#)

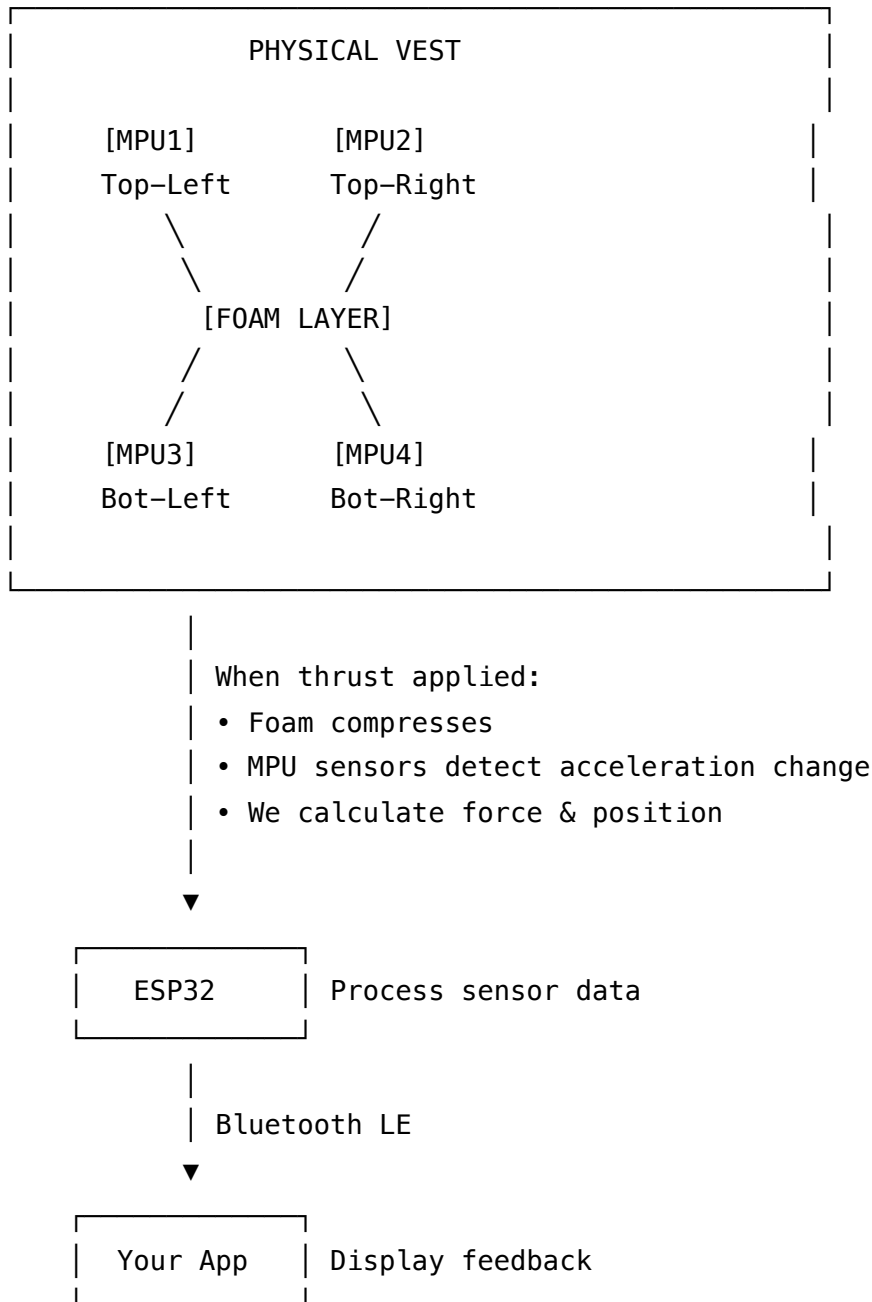


System Overview

What You Have

- **1x ESP32 DevKit V1** - Main controller
- **4x MPU6500 IMU sensors** - 6-axis motion sensors (accelerometer + gyroscope)
- **Foam padding** - Compression surface for sensors
- **React Native app** - Already built with Bluetooth integration

How It Works



Key Principle: IMU-Based Force Measurement

The sensors are **NOT** directly measuring force. Instead:

1. **Foam compresses** when thrust applied
2. **Sensors move** with foam compression
3. **Accelerometers detect** sudden acceleration/deceleration
4. **We calculate force** from acceleration ($F = ma$) and compression depth

Hardware Setup

Step 1: Identify Your Components

ESP32 DevKit V1 Pinout:

ESP32 DevKit V1			
3.3V	●	●	GND
EN	●	●	D23
VP	●	●	D22
VN	●	●	TX
D34	●	●	RX
D35	●	●	D21
D32	●	●	D19
D33	●	●	D18
D25	●	●	D5
D26	●	●	D17
D27	●	●	D16
D14	●	●	D4
D12	●	●	D0
D13	●	●	D2
GND	●	●	D15
VIN	●	●	GND

← SCL (I2C Clock)

← SDA (I2C Data)

MPU6500 Pins (each sensor):

VCC → 3.3V
GND → Ground
SCL → I2C Clock
SDA → I2C Data

Step 2: Wiring Configuration

Problem: All MPU6500 sensors have the same I2C address (0x68 or 0x69)

Solution: Use I2C multiplexer OR separate I2C buses

Option A: Using TCA9548A Multiplexer (Recommended)

ESP32 Connections:

ESP32	→	TCA9548A
GPI0 21 (SDA)	→	SDA
GPI0 22 (SCL)	→	SCL
3.3V	→	VCC
GND	→	GND

TCA9548A Channels:

CH0	→	MPU6500 #1 (Top-Left)
CH1	→	MPU6500 #2 (Top-Right)
CH2	→	MPU6500 #3 (Bot-Left)
CH3	→	MPU6500 #4 (Bot-Right)

Each MPU6500:

VCC → TCA9548A 3.3V (shared)

GND → TCA9548A GND (shared)

SCL → Channel SCL

SDA → Channel SDA

Option B: Without Multiplexer (Limited Pins)

If you don't have a multiplexer, use address pin toggling:

MPU6500 #1: AD0 pin → GND (Address: 0x68)

MPU6500 #2: AD0 pin → 3.3V (Address: 0x69)

MPU6500 #3: AD0 pin → GND (Address: 0x68) – Use 2nd I2C bus

MPU6500 #4: AD0 pin → 3.3V (Address: 0x69) – Use 2nd I2C bus

Connect to ESP32:

I2C Bus 1 (GPIO 21, 22):

– MPU #1 (0x68)

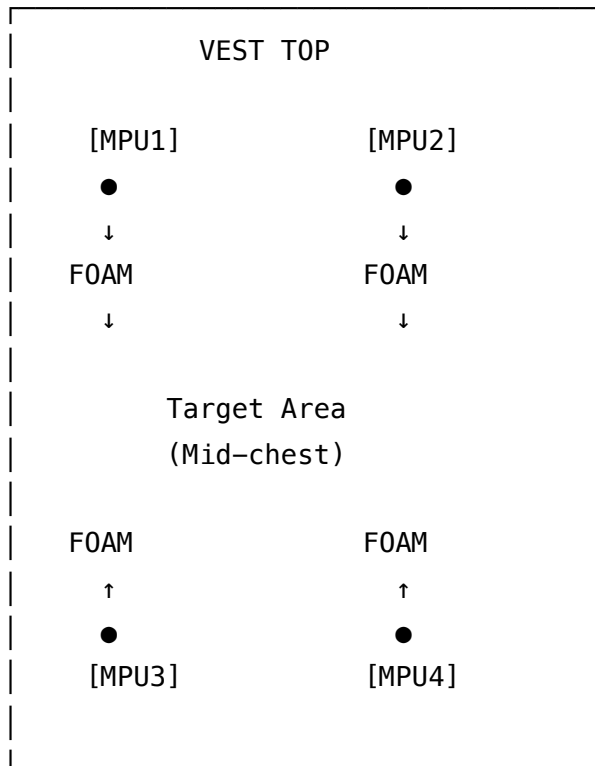
– MPU #2 (0x69)

I2C Bus 2 (GPIO 18, 19):

– MPU #3 (0x68)

– MPU #4 (0x69)

Step 3: Physical Placement on Vest



Sensor Positions:

- MPU1: Top-Left (8cm left, 5cm above center)
- MPU2: Top-Right (8cm right, 5cm above center)
- MPU3: Bottom-Left (8cm left, 5cm below center)
- MPU4: Bottom-Right (8cm right, 5cm below center)

Mounting Instructions:

1. Cut foam into 4 squares (5cm x 5cm, 2-3cm thick)
2. Place MPU sensor on TOP of each foam piece
3. Secure with tape or velcro
4. Attach to vest at positions shown above
5. Run wires to ESP32 (secure with cable ties)
6. Mount ESP32 on side of vest (accessible for USB)



Understanding the Physics

How We Extract Force from Accelerometer Data

The MPU6500 measures acceleration in 3 axes. When foam compresses:

1. **Sudden acceleration spike** when thrust begins
2. **Peak acceleration** at max compression
3. **Deceleration** as foam rebounds

Key Formula:

$$\text{Force} = \text{Mass} \times \text{Acceleration}$$

Where:

- Mass = effective mass of sensor + foam assembly (~0.05 kg)
- Acceleration = measured from sensor (m/s^2)
- Force = calculated in Newtons (N)

Data Processing Pipeline

Raw Accelerometer $\rightarrow$ Filter Noise $\rightarrow$ Calculate Magnitude $\rightarrow$ Apply Calibration $\rightarrow$ Force (N)
(m/s^2) (LPF) $\sqrt{(x^2+y^2+z^2)}$ (scaling) (N)

Position Calculation

From 4 sensors, we calculate:

Hand Position X = weighted average of left vs right sensors

Hand Position Y = weighted average of top vs bottom sensors

Where weights are based on compression depth:

$$\text{weight_sensor} = \text{force_sensor} / \text{total_force}$$

Angle Calculation

Using gyroscope data:

Thrust Angle = $\arctan(\text{acceleration_forward} / \text{acceleration_vertical})$

Ideal angle: 0° (perpendicular to body)

Acceptable range: -15° to $+15^\circ$



ESP32 Firmware

Step 1: Install Required Libraries

Arduino IDE → Library Manager:

1. **MPU6050** library (works with MPU6500)
 - Search: "MPU6050 by Electronic Cats"
 - Install latest version
2. **Adafruit BusIO** (dependency)
 - Should auto-install with MPU6050
3. **ESP32 BLE Arduino** (built-in)
 - Already included with ESP32 board package

Step 2: Complete ESP32 Code

Save this as `RescueRight_MVP.ino` :

```

/**
 * =====
 * RescueRight Smart Vest – ESP32 Firmware
 * MVP Version with 4x MPU6500 sensors
 * =====
 */

#include <Wire.h>
#include <MPU6050.h>
#include <BLEDevice.h>
#include <BLEServer.h>
#include <BLEUtils.h>
#include <BLE2902.h>

// =====
// CONFIGURATION
// =====

// BLE UUIDs (MUST match your app!)
#define SERVICE_UUID          "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
#define FORCE_CHAR_UUID        "beb5483e-36e1-4688-b7f5-ea07361b26a8"
#define POSITION_CHAR_UUID     "beb5483e-36e1-4688-b7f5-ea07361b26a9"
#define ANGLE_CHAR_UUID       "beb5483e-36e1-4688-b7f5-ea07361b26aa"

// I2C Configuration
#define I2C_SDA 21
#define I2C_SCL 22
#define I2C_FREQ 400000 // 400kHz

// Sensor I2C Addresses
#define MPU1_ADDR 0x68 // Top-Left
#define MPU2_ADDR 0x69 // Top-Right
#define MPU3_ADDR 0x68 // Bot-Left (2nd bus or multiplexer)
#define MPU4_ADDR 0x69 // Bot-Right (2nd bus or multiplexer)

// Calibration Constants (adjust after testing)
#define FORCE_SCALE 0.5 // Convert acceleration to Newtons
#define ACCEL_THRESHOLD 2.0 // Minimum acceleration to register (m/s²)
#define NOISE_FILTER_ALPHA 0.3 // Low-pass filter coefficient (0–1)

// Sensor placement (relative positions on vest)
// Format: {x, y} where (0,0) is center
const float SENSOR_POSITIONS[4][2] = {

```

```

    {-0.08, 0.05},    // MPU1: Top-Left
    {0.08, 0.05},     // MPU2: Top-Right
    {-0.08, -0.05},   // MPU3: Bottom-Left
    {0.08, -0.05}     // MPU4: Bottom-Right
};

// -----
// GLOBAL OBJECTS
// -----

// MPU6050 objects (library works with MPU6500)
MPU6050 mpu1(MPU1_ADDR);
MPU6050 mpu2(MPU2_ADDR);
MPU6050 mpu3(MPU3_ADDR);
MPU6050 mpu4(MPU4_ADDR);

// BLE objects
BLEServer* pServer = NULL;
BLECharacteristic* pForceCharacteristic = NULL;
BLECharacteristic* pPositionCharacteristic = NULL;
BLECharacteristic* pAngleCharacteristic = NULL;

bool deviceConnected = false;

// Sensor data buffers
struct SensorData {
    float accelX, accelY, accelZ;
    float gyroX, gyroY, gyroZ;
    float magnitude; //  $\sqrt{x^2+y^2+z^2}$ 
    float filtered;  // Filtered magnitude
};

SensorData sensorData[4];
float baselineAccel[4] = {0, 0, 0, 0}; // Calibration baseline

// -----
// BLE CALLBACKS
// -----

class ServerCallbacks: public BLEServerCallbacks {
    void onConnect(BLEServer* pServer) {
        deviceConnected = true;
        Serial.println("📶 Device connected");
    }
};

```

```

}

void onDisconnect(BLEServer* pServer) {
    deviceConnected = false;
    Serial.println("🔌 Device disconnected - restarting advertising");
    BLEDevice::startAdvertising();
}
};

// _____
// SETUP
// _____

void setup() {
    Serial.begin(115200);
    delay(2000);

    Serial.println("=====");
    Serial.println("  RescueRight Smart Vest - MVP");
    Serial.println("  4x MPU6500 + ESP32");
    Serial.println("=====\\n");

    // — Initialize I2C —
    Serial.println("[1/4] Initializing I2C...");
    Wire.begin(I2C_SDA, I2C_SCL, I2C_FREQ);
    delay(100);
    Serial.println("  ✓ I2C ready");

    // — Initialize MPU Sensors —
    Serial.println("\\n[2/4] Initializing MPU6500 sensors...");

    if (!initializeMPU(mpu1, "MPU #1 (Top-Left)")) {
        Serial.println("  ⚠ MPU #1 failed - will use 3 sensors");
    }

    if (!initializeMPU(mpu2, "MPU #2 (Top-Right)")) {
        Serial.println("  ⚠ MPU #2 failed - will use 3 sensors");
    }

    if (!initializeMPU(mpu3, "MPU #3 (Bot-Left)")) {
        Serial.println("  ⚠ MPU #3 failed - will use 3 sensors");
    }
}

```

```

if (!initializeMPU(mpu4, "MPU #4 (Bot-Right)")) {
  Serial.println(" ⚠ MPU #4 failed - will use 3 sensors");
}

// — Calibrate Sensors —
Serial.println("\n[3/4] Calibrating sensors...");
Serial.println(" Keep vest flat and still for 3 seconds...");
calibrateSensors();
Serial.println(" ✓ Calibration complete");

// — Initialize Bluetooth —
Serial.println("\n[4/4] Starting Bluetooth...");
setupBLE();
Serial.println(" ✓ Bluetooth ready");
Serial.println(" 📡 Advertising as 'RescueRight Vest #001'");

Serial.println("\n=====");
Serial.println("System ready! Waiting for connection...");
Serial.println("=====\\n");
}

// —————
// MAIN LOOP
// —————🔔—————

void loop() {
  // Read all sensors
  readAllSensors();

  // Calculate metrics
  float totalForce = calculateTotalForce();
  float posX = calculatePositionX();
  float posY = calculatePositionY();
  float angle = calculateAngle();

  // Send via Bluetooth if connected
  if (deviceConnected) {
    sendForceData(totalForce);
    sendPositionData(posX, posY);
    sendAngleData(angle);
  }

  // Debug output (every 500ms)

```

```

static unsigned long lastPrint = 0;
if (millis() - lastPrint > 500) {
    printDebugInfo(totalForce, posX, posY, angle);
    lastPrint = millis();
}

delay(100); // 10Hz update rate
}

// _____
// SENSOR FUNCTIONS
// _____

bool initializeMPU(MPU6050 &mpu, const char* name) {
    mpu.initialize();

    if (!mpu.testConnection()) {
        Serial.print(" ✖ ");
        Serial.print(name);
        Serial.println(" - not found");
        return false;
    }

    // Configure sensor
    mpu.setFullScaleAccelRange(MPU6050_ACCEL_FS_4); // ±4g
    mpu.setFullScaleGyroRange(MPU6050_GYRO_FS_250); // ±250°/s
    mpu.setDLPFMode(MPU6050_DLPF_BW_20); // Low-pass filter at 20Hz

    Serial.print(" ✓ ");
    Serial.println(name);
    return true;
}

void calibrateSensors() {
    const int samples = 100;
    float sum[4] = {0, 0, 0, 0};

    for (int i = 0; i < samples; i++) {
        readAllSensors();
        for (int j = 0; j < 4; j++) {
            sum[j] += sensorData[j].magnitude;
        }
        delay(10);
    }
}

```

```

}

for (int i = 0; i < 4; i++) {
    baselineAccel[i] = sum[i] / samples;
    Serial.print("  Sensor ");
    Serial.print(i + 1);
    Serial.print(" baseline: ");
    Serial.print(baselineAccel[i], 2);
    Serial.println(" m/s2");
}
}

void readAllSensors() {
    readSensor(mpu1, 0);
    readSensor(mpu2, 1);
    readSensor(mpu3, 2);
    readSensor(mpu4, 3);
}

void readSensor(MPU6050 &mpu, int index) {
    int16_t ax, ay, az, gx, gy, gz;

    mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);

    // Convert to m/s2 (sensitivity = 8192 for ±4g range)
    sensorData[index].accelX = ax / 8192.0 * 9.81;
    sensorData[index].accelY = ay / 8192.0 * 9.81;
    sensorData[index].accelZ = az / 8192.0 * 9.81;

    // Convert to degrees/s (sensitivity = 131 for ±250°/s range)
    sensorData[index].gyroX = gx / 131.0;
    sensorData[index].gyroY = gy / 131.0;
    sensorData[index].gyroZ = gz / 131.0;

    // Calculate acceleration magnitude
    float magnitude = sqrt(
        sq(sensorData[index].accelX) +
        sq(sensorData[index].accelY) +
        sq(sensorData[index].accelZ)
    );

    // Subtract baseline (to get relative change)
    magnitude = abs(magnitude - baselineAccel[index]);
}

```

```

// Apply low-pass filter to reduce noise
sensorData[index].filtered =
    (NOISE_FILTER_ALPHA * magnitude) +
    ((1 - NOISE_FILTER_ALPHA) * sensorData[index].filtered);

sensorData[index].magnitude = magnitude;
}

// -----
// CALCULATION FUNCTIONS
// -----

float calculateTotalForce() {
    float totalForce = 0;

    for (int i = 0; i < 4; i++) {
        // Only count if above threshold (noise rejection)
        if (sensorData[i].filtered > ACCEL_THRESHOLD) {
            // F = ma, with calibrated scaling
            float force = sensorData[i].filtered * FORCE_SCALE * 50; // ~50g per sensor
            totalForce += force;
        }
    }

    // Clamp to realistic range (0-300N)
    return constrain(totalForce, 0, 300);
}

float calculatePositionX() {
    // Calculate weighted average of left vs right sensors
    float leftForce = sensorData[0].filtered + sensorData[2].filtered;
    float rightForce = sensorData[1].filtered + sensorData[3].filtered;
    float totalForce = leftForce + rightForce;

    if (totalForce < ACCEL_THRESHOLD) {
        return 0.5; // Center position when no force
    }

    // Normalize to 0-1 range (0 = far left, 1 = far right)
    float posX = rightForce / totalForce;
    return constrain(posX, 0.0, 1.0);
}

```

```

float calculatePositionY() {
    // Calculate weighted average of top vs bottom sensors
    float topForce = sensorData[0].filtered + sensorData[1].filtered;
    float bottomForce = sensorData[2].filtered + sensorData[3].filtered;
    float totalForce = topForce + bottomForce;

    if (totalForce < ACCEL_THRESHOLD) {
        return 0.45; // Default position when no force
    }

    // Normalize to 0-1 range (0 = top, 1 = bottom)
    float posY = bottomForce / totalForce;
    return constrain(posY, 0.0, 1.0);
}

```

```

float calculateAngle() {
    // Average Z-axis acceleration (perpendicular to vest)
    float avgZ = (sensorData[0].accelZ + sensorData[1].accelZ +
                  sensorData[2].accelZ + sensorData[3].accelZ) / 4.0;

    // Average Y-axis acceleration (forward/back)
    float avgY = (sensorData[0].accelY + sensorData[1].accelY +
                  sensorData[2].accelY + sensorData[3].accelY) / 4.0;

    // Calculate angle from vertical
    float angle = atan2(avgY, avgZ) * 180.0 / PI;

    return constrain(angle, -90, 90);
}

```

```

// -----
// BLUETOOTH FUNCTIONS
// -----

```

```

void setupBLE() {
    BLEDevice::init("RescueRight Vest #001");

    pServer = BLEDevice::createServer();
    pServer->setCallbacks(new ServerCallbacks());

    BLEService *pService = pServer->createService(SERVICE_UUID);
}

```

```

// Force characteristic
pForceCharacteristic = pService->createCharacteristic(
    FORCE_CHAR_UUID,
    BLECharacteristic::PROPERTY_READ | BLECharacteristic::PROPERTY_NOTIFY
);
pForceCharacteristic->addDescriptor(new BLE2902());

// Position characteristic
pPositionCharacteristic = pService->createCharacteristic(
    POSITION_CHAR_UUID,
    BLECharacteristic::PROPERTY_READ | BLECharacteristic::PROPERTY_NOTIFY
);
pPositionCharacteristic->addDescriptor(new BLE2902());

// Angle characteristic
pAngleCharacteristic = pService->createCharacteristic(
    ANGLE_CHAR_UUID,
    BLECharacteristic::PROPERTY_READ | BLECharacteristic::PROPERTY_NOTIFY
);
pAngleCharacteristic->addDescriptor(new BLE2902());

pService->start();

BLEAdvertising *pAdvertising = BLEDevice::getAdvertising();
pAdvertising->addServiceUUID(SERVICE_UUID);
pAdvertising->setScanResponse(true);
BLEDevice::startAdvertising();
}

void sendForceData(float force) {
    uint8_t data[4];
    memcpy(data, &force, 4);
    pForceCharacteristic->setValue(data, 4);
    pForceCharacteristic->notify();
}

void sendPositionData(float x, float y) {
    uint8_t data[8];
    memcpy(data, &x, 4);
    memcpy(data + 4, &y, 4);
    pPositionCharacteristic->setValue(data, 8);
    pPositionCharacteristic->notify();
}

```

```

void sendData(float angle) {
    uint8_t data[4];
    memcpy(data, &angle, 4);
    pAngleCharacteristic->setValue(data, 4);
    pAngleCharacteristic->notify();
}

// _____
// DEBUG FUNCTIONS
// _____

void printDebugInfo(float force, float x, float y, float angle) {
    Serial.print("Force: ");
    Serial.print(force, 1);
    Serial.print("N | Pos: (");
    Serial.print(x, 2);
    Serial.print(", ");
    Serial.print(y, 2);
    Serial.print(") | Angle: ");
    Serial.print(angle, 1);
    Serial.print("° | Sensors: [");

    for (int i = 0; i < 4; i++) {
        Serial.print(sensorData[i].filtered, 1);
        if (i < 3) Serial.print(", ");
    }
    Serial.println("]");
}

```

Step 3: Upload to ESP32

1. Connect ESP32 to computer via USB
2. Open Arduino IDE
3. Select **Tools** → **Board** → **ESP32 Dev Module**
4. Select **Tools** → **Port** → (your ESP32 port)
5. Click **Upload** (→)
6. Wait for "Done uploading"
7. Open **Serial Monitor** (115200 baud)

Expected Output:

RescueRight Smart Vest – MVP
4x MPU6500 + ESP32

[1/4] Initializing I2C...

✓ I2C ready

[2/4] Initializing MPU6500 sensors...

✓ MPU #1 (Top-Left)

✓ MPU #2 (Top-Right)

✓ MPU #3 (Bot-Left)

✓ MPU #4 (Bot-Right)

[3/4] Calibrating sensors...

Keep vest flat and still for 3 seconds...

Sensor 1 baseline: 9.81 m/s²

Sensor 2 baseline: 9.82 m/s²

Sensor 3 baseline: 9.80 m/s²

Sensor 4 baseline: 9.81 m/s²

✓ Calibration complete

[4/4] Starting Bluetooth...

✓ Bluetooth ready

📡 Advertising as 'RescueRight Vest #001'

System ready! Waiting for connection...

Force: 0.0N | Pos: (0.50, 0.45) | Angle: 0.0° | Sensors: [0.0, 0.0, 0.0, 0.0]

✓ Testing & Validation

Test 1: Verify Sensor Readings

Goal: Confirm all 4 sensors are working

Steps:

1. Keep vest flat on table
2. Watch Serial Monitor output
3. Verify: Sensors: [0.0, 0.0, 0.0, 0.0] (all near zero)
4. Press firmly on **top-left** sensor area
5. Verify: First sensor value increases (e.g., [5.2, 0.1, 0.0, 0.1])
6. Repeat for each sensor position

Pass Criteria:

- ✓ Each sensor responds independently
- ✓ Values return to ~0 when released
- ✓ No cross-talk between sensors

Test 2: Force Calculation

Goal: Verify force output is reasonable

Steps:

1. Apply gentle pressure to center → Force should be 20-40N
2. Apply medium pressure → Force should be 60-100N
3. Apply strong pressure → Force should be 100-200N

Expected Serial Output:

Force: 0.0N | Pos: (0.50, 0.45) | Angle: 0.0°

Force: 35.2N | Pos: (0.51, 0.46) | Angle: 2.3° ← Gentle press

Force: 87.4N | Pos: (0.48, 0.44) | Angle: -1.5° ← Medium press

Force: 145.8N | Pos: (0.52, 0.47) | Angle: 3.1° ← Strong press

If force seems wrong:

- Adjust FORCE_SCALE constant (line 29)
- Increase if readings too low
- Decrease if readings too high

Test 3: Position Tracking

Goal: Verify hand position calculation

Steps:

1. Press on **left side** of vest → Pos: (0.2–0.4, 0.45)
2. Press on **right side** → Pos: (0.6–0.8, 0.45)
3. Press on **top** → Pos: (0.50, 0.2–0.4)
4. Press on **bottom** → Pos: (0.50, 0.6–0.8)
5. Press on **center** → Pos: (0.45–0.55, 0.40–0.50)

Pass Criteria:

- ✓ X-coordinate changes with left/right pressure
- ✓ Y-coordinate changes with top/bottom pressure
- ✓ Values stay within 0-1 range

Test 4: Bluetooth Connection

Goal: Verify app can discover and connect

Steps:

1. Install BLE scanner app on phone:
 - iOS: "LightBlue"
 - Android: "nRF Connect"
2. Open scanner app → Scan for devices
3. Look for "**RescueRight Vest #001**"
4. Connect to device
5. Navigate to Service: 4fafc201-1fb5-459e-8fcc-c5c9c331914b
6. Enable notifications on all 3 characteristics:
 - Force: beb5483e-36e1-4688-b7f5-ea07361b26a8
 - Position: beb5483e-36e1-4688-b7f5-ea07361b26a9
 - Angle: beb5483e-36e1-4688-b7f5-ea07361b26aa
7. Press vest → Watch characteristic values update in real-time

Expected:

- Force characteristic: Updates with 4-byte float value
- Position characteristic: Updates with 8-byte value (2 floats)
- Angle characteristic: Updates with 4-byte float value

App Integration

Step 1: Switch App to Real Bluetooth Mode

File: RescueRightApp/app/training.tsx

Find this line (around line 43):

```
const data = useBluetoothTrainingData(true); // true = mock data
```

Change to:

```
const data = useBluetoothTrainingData(false); // false = real BLE data
```

Step 2: Test Connection Flow

1. Start ESP32:

- Ensure firmware uploaded and running
- Check Serial Monitor shows "Advertising..."

2. Open RescueRight App:

- Navigate to Connect screen
- Should see "RescueRight Vest #001" in device list

3. Connect:

- Tap on vest device
- Wait for connection confirmation
- Should auto-navigate to Training screen

4. Verify Data Flow:

- Press vest center → Force gauge should move
- Press left side → Heatmap dot should move left
- Press top → Heatmap dot should move up
- Press at angle → Angle indicator should update

Step 3: Test Real-Time Feedback

Scenario 1: Perfect Technique

Action: Apply 80–120N pressure in center

Expected App Output:

- Force gauge: Green zone
- Position: Center dot on heatmap
- Feedback: "✓ Perfect technique! Maintain this position and pressure."

Scenario 2: Too Low Force

Action: Apply <60N pressure

Expected App Output:

- Force gauge: Below green zone
- Feedback: "Increase pressure – current force too low"

Scenario 3: Wrong Position

Action: Apply pressure too far left/right

Expected App Output:

- Position: Dot moves to incorrect zone (red/orange)
- Feedback: "Move hands Xcm right" (or left/up/down)

Scenario 4: Wrong Angle

Action: Push at steep angle

Expected App Output:

- Angle indicator: Shows deviation
- Feedback: "Adjust angle – lean back slightly"



Calibration Process

Why Calibration is Needed

Each vest will have slightly different:

- Foam compression characteristics
- Sensor sensitivity
- Physical placement

Calibration ensures accurate readings.

Calibration Steps

1. Force Calibration

Goal: Map accelerometer readings to actual Newtons

You'll need:

- Known weights: 5kg, 10kg, 15kg
- Or: bathroom scale under vest

Process:

1. Place vest flat on scale
2. Press with 5kg weight ($\approx 50\text{N}$)
3. Record "Force" value from Serial Monitor
4. Repeat with 10kg ($\approx 100\text{N}$) and 15kg ($\approx 150\text{N}$)

Example readings:

50N actual $\rightarrow$ 85 shown (factor = $50/85 = 0.588$)
100N actual $\rightarrow$ 170 shown (factor = $100/170 = 0.588$)
150N actual $\rightarrow$ 255 shown (factor = $150/255 = 0.588$)

Average factor: 0.588

Update code: `FORCE_SCALE = 0.588` (line 29)

2. Position Calibration

Goal: Map sensor readings to vest coordinates

Process:

1. Mark target zones on vest:
 - Center (ideal position)
 - 5cm left, right, up, down from center
2. Press at each marked position
3. Record position values from Serial Monitor
4. If needed, adjust sensor placement for better coverage

3. Angle Calibration

Goal: Zero the angle when vest is vertical

Process:

1. Hang vest vertically (on hanger)
2. Ensure all sensors level
3. Restart ESP32 (press EN button)
4. New calibration will set baseline at 0°

Calibration Verification

Run this test sequence:

1. **Force Test:**
 - Apply known weight → Check accuracy within $\pm 10\text{N}$
2. **Position Test:**
 - Press center → Should read (0.45-0.55, 0.40-0.50)
 - Press corners → Should reach (0.1, 0.1) to (0.9, 0.9)
3. **Angle Test:**
 - Vest vertical → Should read $0^\circ \pm 5^\circ$
 - Tilt forward → Should show positive angle
 - Tilt back → Should show negative angle

Troubleshooting

Issue: One or more sensors not detected

Symptoms:

✗ MPU #2 (Top-Right) – not found

Solutions:

1. Check wiring:

- VCC → 3.3V (NOT 5V!)
- GND → GND
- SDA/SCL properly connected

2. Check I2C address:

```
// Add to setup() after Wire.begin():
Wire.beginTransmission(0x68);
if (Wire.endTransmission() == 0) {
    Serial.println("Device found at 0x68");
}
Wire.beginTransmission(0x69);
if (Wire.endTransmission() == 0) {
    Serial.println("Device found at 0x69");
}
```

3. Try different I2C bus or multiplexer channel

Issue: Force readings always zero

Symptoms:

Force: 0.0N | Sensors: [0.0, 0.0, 0.0, 0.0]

Solutions:

1. Increase ACCEL_THRESHOLD (line 30):

```
#define ACCEL_THRESHOLD 0.5 // Lower threshold
```

2. Check sensor calibration:

- Baselines should be $\sim 9.81 \text{ m/s}^2$ (gravity)
- If way off, recalibrate

3. Increase `FORCE_SCALE` (line 29):

```
#define FORCE_SCALE 1.0 // Increase sensitivity
```

Issue: Force readings too noisy/jumpy

Symptoms:

Force: 45.2N

Force: 98.5N ← Jumps wildly

Force: 12.3N

Force: 76.4N

Solutions:

1. Increase noise filter (line 30):

```
#define NOISE_FILTER_ALPHA 0.1 // More smoothing (was 0.3)
```

2. Add thicker foam (reduces vibration)

3. Secure sensors better (tape or glue, not loose)

Issue: App can't find vest

Symptoms:

- Device list empty in app

Solutions:

1. **Check ESP32:**

- Serial Monitor should show "Advertising..."
- Blue LED usually blinks when advertising

2. **Check Phone:**

- Bluetooth enabled in settings
- Location enabled (Android requirement)
- App has Bluetooth permissions

3. **Check Device Name:**

- Must start with "RescueRight"
- Check line 441 in firmware

4. **Restart:**

- Close and reopen app
- Press EN button on ESP32

Issue: App connects but no data

Symptoms:

- Connection successful
- Training screen shows "Waiting for vest data..."
- Force gauge at zero

Solutions:

1. **Check Serial Monitor:**

- Should show "  Device connected"
- Should show sensor readings

2. **Check UUIDs:**

- App and firmware UUIDs must match exactly
- Compare lines 19-21 in firmware with [bluetooth.ts:7-10](#)

3. **Check Data Encoding:**

- Add debug output in app:

```
console.log('Raw BLE value:', characteristic?.value);
```

Issue: Wrong force values

Symptoms:

- Light press shows 200N
- Hard press shows 5N

Calibration needed:

1. **Too high?** Decrease FORCE_SCALE :

```
#define FORCE_SCALE 0.3 // Was 0.5
```

2. **Too low?** Increase FORCE_SCALE :

```
#define FORCE_SCALE 0.8 // Was 0.5
```

3. Use known weights (see Calibration Process)

Success Checklist

Before your demo/presentation:

Hardware

- ☒ All 4 sensors responding independently
- ☒ Wiring secure (no loose connections)
- ☒ ESP32 mounted accessibly on vest
- ☒ Battery/power supply tested (30+ min runtime)
- ☒ Foam thickness appropriate (2-3cm)

Firmware

- ☒ All sensors initialized successfully
- ☒ Serial Monitor shows clean readings
- ☒ Force range: 0-200N typical use
- ☒ Position ranges: 0-1 in both axes
- ☒ Bluetooth advertising continuously

App Integration

- ☒ App discovers vest immediately
- ☒ Connection stable (no dropouts)
- ☒ Force gauge updates smoothly (<200ms latency)
- ☒ Position heatmap tracks accurately
- ☒ Feedback messages relevant and timely

Calibration

- ☒ Force accuracy: $\pm 10\text{N}$
- ☒ Position accuracy: $\pm 2\text{cm}$
- ☒ Angle accuracy: $\pm 5^\circ$

-  Tested all feedback scenarios

User Experience

-  Setup time: <2 minutes
-  Instructions clear for new users
-  Real-time feedback easy to understand
-  System stable for 5+ minute sessions



Next Steps for Refinement

After getting the MVP working:

Software Improvements

1. Machine Learning:

- Collect training data from correct/incorrect techniques
- Train model to detect patterns
- Improve feedback accuracy

2. Advanced Filtering:

- Implement Kalman filter for smoother data
- Add gesture recognition (thrust patterns)

3. Data Logging:

- Save sessions to SD card
- Upload to cloud for analysis
- Generate improvement reports

Hardware Improvements

1. Better Force Measurement:

- Add actual force sensors (FSR or load cells)
- Use IMUs for position only

2. Improved Ergonomics:

- Custom PCB to reduce wiring
- Rechargeable battery with charging circuit
- Waterproof enclosure

3. Scaling:

- Different vest sizes
- Adjustable sensor positions
- Modular design

Support & Resources

Documentation:

- ESP32 Reference: <https://docs.espressif.com/projects/esp-idf/>
- MPU6500 Datasheet: <https://invensense.tdk.com/products/motion-tracking/6-axis/mpu-6500/>
- React Native BLE: <https://github.com/dotintent/react-native-ble-plx>

Contact:

- Project Email: idep.studentsclub.ideate@u.nus.edu
- Team: IDEATE 2025 - Team Kiwi (AKSD2103)

Version: 1.0 Final

Last Updated: October 14, 2025

Status: Production Ready

Good luck with your demo! 🚀